CAPÍTULO

23 MÉTRICAS DE PRODUCTO

Conceptos clave

diseño de interfaz de usuario 545
diseño *webapp* 545
indicador 527
medición 527
medida 527
Meta/Pregunta/Métrica (MPM) 529
métricas
atributos de 530
código fuente 547
diseño arquitectónico 535
diseño OO 537
modelo de requerimientos . 531
orientado a clase 539
pruebas 548
principios de medición 528
punto de función (PF) 531

Un elemento clave de cualquier proceso de ingeniería es la medición. Pueden usarse medidas para entender mejor los atributos de los modelos que se crean y para valorar la calidad de los productos o sistemas sometidos a ingeniería que se construyen. Pero, a diferencia de otras disciplinas de la ingeniería, la del software no está asentada en las leyes cuantitativas de la física. Mediciones directas, como voltaje, masa, velocidad o temperatura, son raras en el mundo del software. Puesto que las mediciones y métricas del software con frecuencia son indirectas, están abiertas a debate. Fenton [Fen91] aborda este conflicto cuando afirma:

> Medición es el proceso mediante el cual se asignan números o símbolos a los atributos de las entidades en el mundo real, de manera que se les define de acuerdo con reglas claramente determinadas [...] En ciencias físicas, medicina, economía y más recientemente en ciencias sociales, ahora es posible medir atributos que anteriormente se consideraban inmensurables [...] Desde luego, tales mediciones no son tan refinadas como muchas mediciones en las ciencias físicas [...], pero existen [y con base en ellas se toman decisiones importantes]. Sentimos que la obligación de intentar "medir lo inmensurable" para mejorar la comprensión de entidades particulares es tan poderosa en la ingeniería del software como en cualquiera otra disciplina.

Pero algunos miembros de la comunidad del software continúan argumentando que el software es "inmensurable" o que los intentos por medir deben posponerse hasta comprender mejor el software y los atributos que deben usarse para describirlo. Esto es un error.

Una mirada rápida

¿Qué es? Por su naturaleza, la ingeniería es una disciplina cuantitativa. Las métricas de producto ayudan a los ingenieros de software a obtener comprensión acerca del diseño y la construcción del software que elaboran, al enfocarse en atributos mensurables específicos de los productos de trabajo de la ingeniería del software.

¿Quién lo hace? Los ingenieros de software usan métricas de proyecto para auxiliarse en la construcción de software de mayor calidad.

¿Por qué es importante? Siempre habrá un elemento cualitativo en la creación del software para computadoras. El problema es que la valoración cualitativa tal vez no sea suficiente. Se necesitan criterios objetivos que ayuden a guiar el diseño de datos, arquitectura, interfaces y componentes. Cuando se prueban, es necesaria la guía cuantitativa que ayuda en la selección de los casos de prueba y de sus objetivos. Las métricas de producto proporcionan una base desde donde el análisis, el diseño, la codificación y las pruebas pueden realizarse de manera más objetiva y valorarse de modo más cuantitativo.

¿Cuáles son los pasos? El primer paso en el proceso de medición es derivar las mediciones y métricas del software que sean adecuadas para la presentación del software que se está construyendo. A continuación, se recolectan los datos requeridos para derivar las métricas formuladas. Una vez calculadas, las métricas adecuadas se analizan con base en lineamientos preestablecidos y en datos anteriores. Los resultados del análisis se interpretan para obtener comprensión acerca de la calidad del software, y los resultados de la interpretación conducen a modificación de requerimientos y modelos de diseño, código fuente o casos de prueba. En algunas instancias, también puede conducir a modificación del proceso de software en sí.

¿Cuál es el producto final? Las métricas de producto que se calculan a partir de los datos recolectados de los modelos de requerimientos y de diseño, código fuente y casos de prueba.

¿Cómo me aseguro de que lo hice bien? Debe establecer los objetivos de la medición antes de comenzar la recolección de datos y debe definir cada métrica de producto sin ambigüedades. Defina sólo algunas métricas y luego úselas para obtener comprensión acerca de la calidad de un producto de trabajo de ingeniería del software.

Aunque las métricas de producto para el software de computadora son imperfectas, pueden proporcionar una forma sistemática de valorar la calidad con base en un conjunto de reglas claramente definidas. También proporcionan comprensión inmediata, en lugar de hacerlo después de los hechos. Esto permite descubrir y corregir potenciales problemas antes de que se conviertan en defectos catastróficos.

En este capítulo se presentan las mediciones que pueden usarse para valorar la calidad del producto conforme se somete a ingeniería. Estas mediciones de atributos internos del producto ofrecen una indicación en tiempo real de la eficacia de los modelos de requerimientos, diseño y código, así como de la efectividad de los casos de prueba y de la calidad global del software que se va a construir.

23.1 Marco conceptual para las métricas de producto

Como se anotó en la introducción, la medición asigna números o símbolos a atributos de entidades en el mundo real. Para lograr esto, se requiere un modelo de medición que abarque un conjunto consistente de reglas. Aunque la teoría de la medición (por ejemplo, [Kyb84]) y su aplicación al software de computadora (por ejemplo, [Zus97]) son temas que están más allá del ámbito de este libro, vale la pena establecer un marco conceptual fundamental y un conjunto de principios básicos que guíen la definición de las métricas de producto para el software.

Cita:

"Una ciencia es tan madura como sus herramientas de medición."

Louis Pasteur

23.1.1 Medidas, métricas e indicadores

Aunque los términos *medida, medición* y *métrica* con frecuencia se usan de modo intercambiable, es importante observar las sutiles diferencias entre ellos. En el contexto de la ingeniería del software, una *medida* proporciona un indicio cuantitativo de la extensión, cantidad, dimensión, capacidad o tamaño de algún atributo de un producto o proceso. La *medición* es el acto de determinar una medida. El *IEEE Standard Glosary of Software Engineering Terminology* [IEE93b] define *métrica* como "una medida cuantitativa del grado en el que un sistema, componente o proceso posee un atributo determinado".

¿Cuál es la diferencia entre una medida y una métrica?

Cuando se ha recolectado un solo punto de datos (por ejemplo, el número de errores descubiertos dentro de un solo componente de software), se establece una medida. La medición ocurre como resultado de la recolección de uno o más puntos de datos (por ejemplo, algunas revisiones de componente y pruebas de unidad se investigan para recolectar medidas del número de errores de cada uno). Una métrica de software relaciona en alguna forma las medidas individuales (por ejemplo, el número promedio de errores que se encuentran por revisión o el número promedio de errores que se encuentran por unidad de prueba).

PUNTO CLAVE

Un indicador es una o varias métricas que proporcionan comprensión acerca del proceso, el producto o el proyecto.

Un ingeniero de software recolecta medidas y desarrolla métricas de modo que se obtengan indicadores. Un *indicador* es una métrica o combinación de métricas que proporcionan comprensión acerca del proceso de software, el proyecto de software o el producto en sí. Un indicador proporciona comprensión que permite al gerente de proyecto o a los ingenieros de software ajustar el proceso, el proyecto o el producto para hacer mejor las cosas.

23.1.2 El reto de la métrica de producto

Durante las cuatro décadas pasadas, muchos investigadores intentaron desarrollar una sola métrica que proporcionara una medida abarcadora de la complejidad del software. Fenton [Fen94] caracteriza esta investigación como una búsqueda del "imposible Santo Grial". Aunque se han propuesto decenas de medidas de complejidad [Zus90], cada una toma una visión un poco diferente de lo que es la complejidad y de qué atributos de un sistema conducen a la complejidad. Por analogía, considere una métrica para evaluar un automóvil atractivo. Algunos observadores pueden enfatizar el diseño de la carrocería; otros pueden considerar característi-

Cita:

"Así como la medición de temperatura comenzó con un dedo índice... y creció hasta escalas, herramientas y técnicas sofisticadas, de igual modo madura la medición del software."

Shari Pfleeger

cas mecánicas; otros más pueden destacar el costo o el rendimiento o el uso de combustibles alternativos o la capacidad para reciclar cuando el carro sea chatarra. Dado que cualquiera de estas características puede estar en desacuerdo con otras, es difícil derivar un solo valor para el "atractivo". El mismo problema ocurre con el software de computadora.

Aunque existe la necesidad de medir y de controlar la complejidad del software, y si bien un solo valor de esta métrica de calidad es difícil de derivar, es posible desarrollar medidas de diferentes atributos internos de programa (por ejemplo, modularidad efectiva, independencia funcional y otros atributos estudiados en el capítulo 8). Estas medidas y las métricas derivadas de ellas pueden usarse como indicadores independientes de la calidad de los modelos de requerimientos y de diseño. Pero aquí de nuevo surgen problemas. Fenton [Fen94] observa esto cuando afirma: "el peligro de intentar encontrar medidas que caracterizan tantos atributos diferentes es que, inevitablemente, las medidas tienen que satisfacer objetivos en conflicto. Esto es contrario a la teoría representacional de las mediciones". Aunque el enunciado de Fenton es correcto, muchas personas argumentan que la medición de producto realizada durante las primeras etapas del proceso de software brinda a los ingenieros de software un mecanismo consistente y objetivo para valorar la calidad.

WebRef

Horst Zuse compiló voluminosa información acerca de la métrica de producto en **irb.cs.tu-berlin.de/~zuse/**

Sin embargo, es justo preguntar cuán válidas son las métricas de producto, es decir, ¿cuán cercanamente se alinean las métricas de producto con la confiabilidad a largo plazo y con la calidad de un sistema basado en computadora? Fenton [Fen91] aborda esta pregunta en la forma siguiente:

> A pesar de las conexiones intuitivas entre la estructura interna de los productos de software [métricas de producto] y sus atributos externos de producto y proceso, en realidad ha habido pocos intentos científicos para establecer relaciones específicas. Existen algunas razones por las que esto es así; la más comúnmente citada es lo impráctico que resulta realizar experimentos relevantes.

Cada uno de los "retos" anotados aquí es un motivo de precaución, pero no es razón para desechar las métricas de producto.[1] La medición es esencial si debe lograrse la calidad.

23.1.3 Principios de medición

Antes de presentar una serie de métricas de producto que 1) auxilien en la evaluación de los modelos de análisis y diseño, 2) proporcionen un indicio de la complejidad de los diseños procedimentales y del código fuente y 3) faciliten el diseño de pruebas más efectivas, es importante comprender los principios de medición básicos. Roche [Roc94] sugiere un proceso de medición que puede caracterizarse mediante cinco actividades:

? ¿Cuáles son los pasos de un proceso de medición efectivo?

- *Formulación.* La derivación de medidas y métricas de software apropiadas para la representación del software que se está construyendo.
- *Recolección.* Mecanismo que se usa para acumular datos requeridos para derivar las métricas formuladas.
- *Análisis.* El cálculo de métricas y la aplicación de herramientas matemáticas.
- *Interpretación.* Evaluación de las métricas resultantes para comprender la calidad de la representación.
- *Retroalimentación.* Recomendaciones derivadas de la interpretación de las métricas del producto, transmitidas al equipo de software.

1 Aunque las críticas de métricas específicas son comunes en la literatura, muchos críticos se enfocan en conflictos particulares y pierden el objetivo principal de las métricas en el mundo real: ayudar al ingeniero de software a establecer una vía sistemática y objetiva para obtener comprensión de su trabajo y mejorar la calidad del producto resultante.

Las métricas de software serán útiles sólo si se caracterizan efectivamente y si se validan de manera adecuada. Los siguientes principios [Let03b] son representativos de muchos que pueden proponerse para la caracterización y validación de métricas:



En realidad, muchas métricas de producto actualmente en uso no concuerdan con dichos principios tanto como debieran. Pero eso no significa que no tengan valor; sólo tenga cuidado cuando los use y entienda que tienen la intención de proporcionar comprensión, no estricta verificación científica.

- *Una métrica debe tener propiedades matemáticas deseables,* es decir, el valor de la métrica debe estar en un rango significativo (por ejemplo, 0 a 1, donde 0 realmente significa ausencia, 1 indica el valor máximo y 0.5 representa el "punto medio"). Además, una métrica que intente estar en una escala racional no debe constituirse con componentes que sólo se miden en una escala ordinal.
- *Cuando una métrica representa una característica de software que aumenta cuando ocurren rasgos positivos o que disminuye cuando se encuentran rasgos indeseables, el valor de la métrica debe aumentar o disminuir en la misma forma.*
- *Cada métrica debe validarse de manera empírica en una gran variedad de contextos antes de publicarse o utilizarse para tomar decisiones.* Una métrica debe medir el factor de interés, independientemente de otros factores. Debe "escalar" a sistemas más grandes y funcionar en varios lenguajes de programación y dominios de sistema.

Aunque la formulación, caracterización y validación son cruciales, la recolección y el análisis son las actividades que impulsan el proceso de medición. Roche [Roc94] sugiere los siguientes principios para dichas actividades: 1) siempre que sea posible, la recolección y el análisis de datos deben automatizarse; 2) deben aplicarse técnicas estadísticas válidas para establecer relaciones entre atributos de producto internos y características de calidad externas (por ejemplo, si el nivel de complejidad arquitectónica se correlaciona con el número de defectos reportados en el uso de producción), y 3) para cada métrica deben establecerse lineamientos y recomendaciones interpretativos.

23.1.4 Medición de software orientado a meta

WebRef

En **www.thedacts.com/GoldPractices/practices/gqma.html** puede encontrar un útil foro de discusión sobre el MPM.

El paradigma *Meta/Pregunta/Métrica* (MPM) fue desarrollado por Basili y Weiss [Bas84] como una técnica para identificar métricas significativas para cualquier parte del proceso de software. MPM enfatiza la necesidad de: 1) establecer una *meta* de medición explícita que sea específica para la actividad del proceso o para la característica del producto que se quiera valorar, 2) definir un conjunto de *preguntas* que deban responderse con la finalidad de lograr la meta y 3) identificar *métricas* bien formuladas que ayuden a responder dichas preguntas.

Para definir cada meta de medición, puede usarse una *plantilla de definición de meta* [Bas94]. La plantilla toma la forma:

> **Analizar** {el nombre de la actividad o atributo que se va a medir} **con el propósito de** {el objetivo global del análisis[2]} **con respecto a** {el aspecto de la actividad o atributo que se considera} **desde el punto de vista de** {las personas que tienen interés en la medición} **en el contexto de** {el entorno en el que tiene lugar la medición}.

Como ejemplo, considere una plantilla de definición de meta para *CasaSegura*:

> **Analizar** la arquitectura del software *CasaSegura* **con el propósito de** evaluar los componentes arquitectónicos **con respecto a** la capacidad de hacer *CasaSegura* más extensible **desde el punto de vista de** los ingenieros de software que realizan el trabajo **en el contexto de** mejora del producto durante los próximos tres años.

2 Van Solingen y Berghout [Sol90] sugieren que el objeto casi siempre es "comprender, controlar o mejorar" la actividad del proceso o el atributo del producto.

Con una meta de medición definida de manera explícita, se desarrolla un conjunto de preguntas. Las respuestas ayudarán al equipo de software (o a otros participantes) a determinar si se logró la meta de medición. Entre las preguntas que pueden plantearse se encuentran:

P_1: ¿Los componentes arquitectónicos se caracterizan de forma que compartimentalizan la función y los datos relacionados?

P_2: ¿La complejidad de cada componente dentro de las fronteras facilitará la modificación y la extensión?

Cada una de estas preguntas debe responderse de manera cuantitativa, usando una o más medidas y métricas. Por ejemplo, una métrica que proporciona un indicio de la cohesión (capítulo 8) de un componente arquitectónico puede ser útil para responder P_1. Las métricas que se estudian más adelante en este capítulo pueden proporcionar comprensión para P_2. En todo caso, las métricas que se eligen (o derivan) deben corresponderse con los principios de medición analizados en la sección 23.1.3 y con los atributos de medición expuestos en la sección 23.1.5.

23.1.5 Atributos de las métricas de software efectivas

Se han propuesto cientos de métricas para el software de computadora, pero no todas brindan apoyo práctico al ingeniero de software. Algunas demandan medición demasiado compleja, otras son tan particulares que pocos profesionales del mundo real tienen alguna esperanza de entenderlas y otras más violan las nociones intuitivas básicas de lo que realmente es el software de alta calidad.

Ejiogu [Eji91] define un conjunto de atributos que deben abarcar las métricas de software efectivas. La métrica derivada y las medidas que conducen a ella deben ser:

¿Cómo se valora la calidad de una métrica de software propuesta?

- *Simple y calculable.* Debe ser relativamente fácil aprender cómo derivar la métrica y su cálculo no debe demandar esfuerzo o tiempo excesivo.
- *Empírica e intuitivamente convincente.* Debe satisfacer las nociones intuitivas del ingeniero acerca del atributo de producto que se elabora (por ejemplo, una métrica que mide la cohesión del módulo debe aumentar en valor conforme aumenta el nivel de cohesión).
- *Congruente y objetiva.* Siempre debe producir resultados que no tengan ambigüedades. Una tercera parte independiente debe poder derivar el mismo valor de métrica usando la misma información acerca del software.
- *Constante en su uso de unidades y dimensiones.* El cálculo matemático de la métrica debe usar medidas que no conduzcan a combinaciones extrañas de unidades. Por ejemplo, multiplicar personas en los equipos de proyecto por variables de lenguaje de programación en el programa da como resultado una mezcla sospechosa de unidades que no son intuitivamente convincentes.
- *Independiente del lenguaje de programación.* Debe basarse en el modelo de requerimientos, el modelo de diseño o la estructura del programa en sí. No debe depender de los caprichos de la sintaxis o de la semántica del lenguaje de programación.
- *Un mecanismo efectivo para retroalimentación de alta calidad.* Debe proporcionar información que pueda conducir a un producto final de mayor calidad.

CONSEJO

La experiencia indica que una métrica de producto sólo se usará si es intuitiva y fácil de calcular. Si se tienen que hacer decenas de "conteos" y se requieren cálculos complejos, es improbable que la métrica se adopte ampliamente.

Aunque la mayoría de las métricas de software satisfacen estos atributos, algunas métricas de uso común pueden fracasar para satisfacer uno o dos de ellos. Un ejemplo es el punto de función (que se estudia en la sección 23.2.1), una medida de la "funcionalidad" entregada por el software. Puede argumentarse[3] que el atributo de congruente y objetiva fracasa porque una tercera

3 Puede plantearse un contrargumento igualmente vigoroso. Tal es la naturaleza de las métricas del software.

parte independiente no puede ser capaz de derivar el mismo valor del punto de función que un colega que use la misma información acerca del software. Por tanto, ¿debe rechazarse la medida PF? La respuesta es "¡desde luego que no!". El PF proporciona comprensión útil y, en consecuencia, ofrece distinto valor, incluso si falla en satisfacer un atributo a la perfección.

CASASEGURA



Debate acerca de las métricas de producto

La escena: Cubículo de Vinod.

Participantes: Vinod, Jamie y Ed, miembros del equipo de ingeniería del software *CasaSegura*, quienes continúan trabajando en el diseño en el nivel de componentes y en el diseño de casos de prueba.

La conversación:

Vinod: Doug [Doug Miller, gerente de ingeniería del software] me dijo que todos debemos usar métricas de producto, pero fue muy vago. También dijo que no presionaría... que usarlas era asunto nuestro.

Jamie: Eso está bien porque no hay forma de que yo tenga tiempo para comenzar esa cosa de las medidas. Estamos peleando por mantener el calendario como está.

Ed: Estoy de acuerdo con Jamie. Estamos en contra, aquí... no tenemos tiempo.

Vinod: Sí, lo sé, pero probablemente hay algún mérito en usarlas.

Jamie: No lo discuto, Vinod, es cuestión de tiempo... y, en lo que a mí respecta, no tengo para perderlo.

Vinod: Pero, ¿y si las mediciones te ahorran tiempo?

Ed: Estás mal, requieren tiempo, y, como dijo Jamie...

Vinod: No, espera... ¿y si nos ahorra tiempo?

Jamie: ¿Cómo?

Vinod: Volver a trabajar... así es cómo. Si una medida que usemos nos ayuda a evitar un problema grande o incluso moderado, y esto evita que tengamos que volver a trabajar una parte del sistema, ahorramos tiempo. ¿O no?

Ed: Es posible, supongo, ¿pero puedes garantizarnos que alguna métrica de producto nos ayudará a encontrar un problema?

Vinod: ¿Puedes garantizarme que no lo hará?

Jamie: ¿Y qué es lo que propones?

Vinod: Creo que debemos seleccionar algunas métricas de diseño, probablemente orientadas a clase, y usarlas como parte de nuestro proceso de revisión para cada componente que desarrollemos.

Ed: No estoy familiarizado con las métricas orientadas a clase.

Vinod: Yo pasaré algo de tiempo revisándolas y haré una recomendación... ¿está bien para ustedes?

[Ed y Jamie asienten sin mucho entusiasmo.]

23.2 MÉTRICAS PARA EL MODELO DE REQUERIMIENTOS

El trabajo técnico en la ingeniería del software comienza con la creación del modelo de requerimientos. En esta etapa se derivan los requerimientos y se establece un cimiento para el diseño. Por tanto, son deseables métricas de producto que proporcionen comprensión acerca de la calidad del modelo de análisis.

Aunque en la literatura han aparecido relativamente pocas métricas de análisis y especificación, es posible adaptar las métricas que se usan frecuentemente para estimación de proyectos y aplicarlas en este contexto. Dichas métricas examinan el modelo de requerimientos con la intención de predecir el "tamaño" del sistema resultante. En ocasiones (mas no siempre), el tamaño es un indicador de la complejidad del diseño y casi siempre es un indicador creciente de codificación, integración y esfuerzo de pruebas.

23.2.1 Métrica basada en funciones

La *métrica de punto de función* (PF) puede usarse de manera efectiva como medio para medir la funcionalidad que entra a un sistema.[4] Al usar datos históricos, la métrica PF puede entonces usarse para: 1) estimar el costo o esfuerzo requerido para diseñar, codificar y probar el software;

4 Acerca de las métricas PF se han escrito cientos de libros, ensayos y artículos. En [IFP05] puede encontrar una valiosa bibliografía.

WebRef

En **www.ifpug.org** y en **www.functionpoints.com** puede obtenerse mucha información útil acerca de los puntos de función.

2) predecir el número de errores que se encontrarán durante las pruebas, y 3) prever el número de componentes y/o de líneas fuente proyectadas en el sistema implementado.

Los puntos de función se derivan usando una relación empírica basada en medidas contables (directas) del dominio de información del software y en valoraciones cualitativas de la complejidad del software. Los valores de dominio de información se definen en la forma siguiente:[5]

Número de entradas externas (EE). Cada *entrada externa* se origina de un usuario o se transmite desde otra aplicación, y proporciona distintos datos orientados a aplicación o información de control. Con frecuencia, las entradas se usan para actualizar *archivos lógicos internos* (ALI). Las entradas deben distinguirse de las consultas, que se cuentan por separado.

Número de salidas externas (SE). Cada *salida externa* es datos derivados dentro de la aplicación que ofrecen información al usuario. En este contexto, salida externa se refiere a reportes, pantallas, mensajes de error, etc. Los ítems de datos individuales dentro de un reporte no se cuentan por separado.

Número de consultas externas (CE). Una *consulta externa* se define como una entrada en línea que da como resultado la generación de alguna respuesta de software inmediata en la forma de una salida en línea (con frecuencia recuperada de un ALI).

Número de archivos lógicos internos (ALI). Cada *archivo lógico interno* es un agrupamiento lógico de datos que reside dentro de la frontera de la aplicación y se mantiene mediante entradas externas.

Número de archivos de interfaz externos (AIE). Cada *archivo de interfaz externo* es un agrupamiento lógico de datos que reside fuera de la aplicación, pero que proporciona información que puede usar la aplicación.

Una vez recolectados dichos datos, la tabla de la figura 23.1 se completa y un valor de complejidad se asocia con cada conteo. Las organizaciones que usan métodos de punto de función desarrollan criterios para determinar si una entrada particular es simple, promedio o compleja. No obstante, la determinación de complejidad es un tanto subjetiva.

Para calcular puntos de función (PF), se usa la siguiente relación:

$$\text{PF} = \text{conteo total} \times [0.65 + 0.01 \times \Sigma (F_i)] \tag{23.1}$$

donde conteo total es la suma de todas las entradas PF obtenidas de la figura 23.1.

Los F_i (i = 1 a 14) son *factores de ajuste de valor* (FAV) con base en respuestas a las siguientes preguntas [Lon02]:

FIGURA 23.1

Cálculo de puntos de función

Valor de dominio de información	Conteo		Factor ponderado Simple	Promedio	Complejo		
Entradas externas (EE)		×	3	4	6	=	
Salidas externas (SE)		×	4	5	7	=	
Consultas externas (CE)		×	3	4	6	=	
Archivos lógicos internos (ALI)		×	7	10	15	=	
Archivos de interfaz externos (AIE)		×	5	7	10	=	
Conteo total	→						

5 En realidad, la definición de valores de dominio de información y la forma en la que se cuentan son un poco más complejos. Para más detalles, el lector interesado debe consultar [IFP01].

PUNTO CLAVE

Los factores de ajuste de valor se usan para proporcionar un indicio de la complejidad del problema.

1. ¿El sistema requiere respaldo y recuperación confiables?
2. ¿Se requieren comunicaciones de datos especializadas para transferir información hacia o desde la aplicación?
3. ¿Existen funciones de procesamiento distribuidas?
4. ¿El desempeño es crucial?
5. ¿El sistema correrá en un entorno operativo existente enormemente utilizado?
6. ¿El sistema requiere entrada de datos en línea?
7. ¿La entrada de datos en línea requiere que la transacción de entrada se construya sobre múltiples pantallas u operaciones?
8. ¿Los ALI se actualizan en línea?
9. ¿Las entradas, salidas, archivos o consultas son complejos?
10. ¿El procesamiento interno es complejo?
11. ¿El código se diseña para ser reutilizable?
12. ¿La conversión y la instalación se incluyen en el diseño?
13. ¿El sistema se diseña para instalaciones múltiples en diferentes organizaciones?
14. ¿La aplicación se diseña para facilitar el cambio y su uso por parte del usuario?

WebRef

En **irb.cs.uni-mogdeburg.de/sw-eng/us/java/fp/** puede encontrar una calculadora PF en línea.

Cada una de estas preguntas se responde usando una escala que varía de 0 (no importante o aplicable) a 5 (absolutamente esencial). Los valores constantes en la ecuación (23.1) y los factores ponderados que se aplican a los conteos de dominio de información se determinan de manera empírica.

Para ilustrar el uso de la métrica PF en este contexto, considere la representación simple de modelo de análisis que se ilustra en la figura 23.2. En la figura, se representa un diagrama de flujo de datos (capítulo 7) para una función dentro del software *CasaSegura*.

La función gestiona la interacción del usuario, acepta la contraseña de éste para activar o desactivar el sistema y permite consultas sobre el estado de las zonas de seguridad y de varios sensores de seguridad. La función despliega una serie de mensajes de advertencia y envía señales de control adecuadas a varios componentes del sistema de seguridad.

El diagrama de flujo de datos se evalúa para determinar un conjunto de medidas de dominio de información clave que son requeridas para calcular la métrica de punto de función. En la figura se muestran tres entradas externas (**contraseña**, **botón de pánico** y **activar/desactivar**), junto con dos consultas externas (**consulta de zona** y **consulta de sensor**). Se muestra un ALI (**archivo configuración sistema**) y también están presentes dos salidas externas

FIGURA 23.2

Modelo de flujo de datos para el software *CasaSegura*

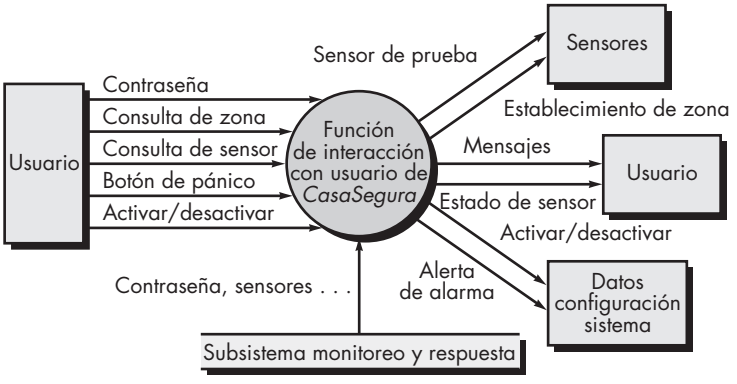


FIGURA 23.3
Cálculo de puntos de función

Valor dominio de información	Conteo		Factor ponderado: Simple	Promedio	Complejo		
Entradas externas (EE)	3	×	3	4	6	=	9
Salidas externas (SE)	2	×	4	5	7	=	8
Consultas externas (CE)	2	×	3	4	6	=	6
Archivos lógicos internos (ALI)	1	×	7	10	15	=	7
Archivos de interfaz externos (AIE)	4	×	5	7	10	=	20
Conteo total							50

(**mensajes** y **estado de sensor**) y cuatro AIE (**sensor de prueba**, **establecimiento de zona**, **activar/desactivar** y **alerta de alarma**). En la figura 23.3 se muestran estos datos, junto con la complejidad adecuada.

El conteo total que se muestra en la figura 23.3 debe ajustarse usando la ecuación (23.1). Para los propósitos de este ejemplo, suponga que $\Sigma(F_i)$ es 46 (un producto moderadamente complejo). Por tanto,

$$PF = 50 \times [0.65 + (0.01 \times 46)] = 56$$

Cita:

"En lugar de sólo meditar acerca de cuál 'nueva métrica' aplicar [...] también debemos plantearnos la pregunta más básica: ¿qué haremos con las métricas?"

Michael Mah y Larry Putnam

Con base en el valor PF proyectado, derivado del modelo de requerimientos, el equipo del proyecto puede estimar el tamaño global implementado de la función de interacción del usuario de *CasaSegura*. Suponga que los datos anteriores indican que un PF se traduce en 60 líneas de código (se usará un lenguaje orientado a objeto) y que se producen 12 PF por cada persona-mes de esfuerzo. Estos datos históricos ofrecen al gerente de proyecto información importante de planificación que se basa en el modelo de requerimientos y no en estimaciones preliminares. Suponga aún más, que en los proyectos pasados se encontró un promedio de tres errores por punto de función durante las revisiones de requerimientos y diseño, y cuatro errores por punto de función durante las pruebas de unidad e integración. A final de cuentas, dichos datos pueden ayudarlo a valorar lo completo de sus actividades de revisión y pruebas.

Uemura *et al.* [Uem99] sugieren que los puntos de función también pueden calcularse a partir de clases UML y diagramas de secuencia. Si tiene más interés, consulte detalles en [Uem99].

23.2.2 Métricas para calidad de la especificación

Davis *et al.* [Dav93] proponen una lista de características que pueden usarse para valorar la calidad del modelo de requerimientos y la correspondiente especificación de requerimientos: *especificidad* (falta de ambigüedad), *completitud*, *corrección*, *comprensibilidad*, *verificabilidad*, *consistencia interna y externa*, *factibilidad*, *concisión*, *rastreabilidad*, *modificabilidad*, *precisión* y *reusabilidad*. Además, los autores observan que las especificaciones de alta calidad se almacenan electrónicamente, son ejecutables o al menos interpretables, se anotan mediante importancia relativa, son estables, tienen versión, se organizan, cuentan con referencia cruzada y se especifican en el nivel correcto de detalle.

Aunque muchas de estas características parecen ser cualitativas por naturaleza, Davis *et al.* [Dav93] sugieren que cada una puede representarse usando una o más métricas. Por ejemplo, se supone que existen n_r requerimientos en una especificación, tales que

$$n_r = n_f + n_{nf}$$

donde n_f es el número de requerimientos funcionales y n_{nf} es el número de requerimientos no funcionales (por ejemplo, rendimiento).

PUNTO CLAVE

Al medir las características de la especificación, es posible obtener comprensión cuantitativa acerca de la especificidad y de la completitud.

Para determinar la *especificidad* (falta de ambigüedad) de los requerimientos, Davis *et al.*, sugieren una métrica que se basa en la consistencia de la interpretación de los revisores de cada requisito:

$$Q_1 = \frac{n_{ui}}{n_r}$$

donde n_{ui} es el número de requerimientos para los cuales todos los revisores tienen interpretaciones idénticas. Mientras más cercano a 1 esté el valor de Q, menor será la ambigüedad de la especificación.

La *completitud* de los requerimientos funcionales puede determinarse al calcular la razón

$$Q_2 = \frac{n_u}{n_i \times n_s}$$

donde n_u es el número de requerimientos funcionales únicos, n_i es el número de entradas (estímulos) definidas o implicadas por la especificación y n_s es el número de estados especificados. La razón Q_2 mide el porcentaje de funciones necesarias que se especificaron para un sistema. Sin embargo, no aborda requerimientos no funcionales. Para incorporar éstos en una métrica global de completitud, se debe considerar el grado en el que se validaron los requerimientos:

Cita:

"Medir lo que es mensurable y lo que no es mensurable, hace [lo] mensurable."

Galileo

$$Q_3 = \frac{n_c}{n_c + n_{nv}}$$

donde n_c es el número de requerimientos que se validaron como correctos y n_{nv} es el número de requerimientos que no se han validado.

23.3 MÉTRICAS PARA EL MODELO DE DISEÑO

PUNTO CLAVE

Las métricas pueden proporcionar comprensión acerca de los datos estructurales y de la complejidad del sistema asociada con el diseño arquitectónico.

Es inconcebible que el diseño de una nueva aeronave, un nuevo chip de computadora o un nuevo edificio de oficinas se realizara sin definir medidas de diseño, ni determinar las métricas para varios aspectos de la calidad del diseño ni usarlos como indicadores para guiar la forma en la que evoluciona el diseño. Y aún así, con frecuencia el diseño de los sistemas complejos basados en software procede virtualmente sin medición. La ironía de esto es que están disponibles métricas del diseño para el software, pero la gran mayoría de los ingenieros del software continúan sin percatarse de su existencia.

Las métricas de diseño para software de computadora, al igual que todas las demás métricas de software, no son perfectas. El debate continúa acerca de su eficacia y sobre la forma en la que deben aplicarse. Muchos expertos argumentan que se requiere más experimentación antes de poder usar las medidas de diseño, aunque el diseño sin medición es una alternativa inaceptable.

En las siguientes secciones se examinan algunas de las métricas de diseño más comunes para software de computadora. Cada una puede proporcionarle comprensión mejorada y todas pueden ayudar a que el diseño evolucione hacia un mayor nivel de calidad.

23.3.1 Métricas del diseño arquitectónico

Las métricas del diseño arquitectónico se enfocan en características de la arquitectura del programa (capítulo 9) con énfasis en la estructura arquitectónica y en la efectividad de los módulos o componentes dentro de la arquitectura. Dichas métricas son "caja negra" en tanto no requieren conocimiento alguno del funcionamiento interior de un componente de software particular.

Card y Glass [Car90] definen tres medidas de complejidad del diseño de software: complejidad estructural, complejidad de datos y complejidad del sistema.

Para arquitecturas jerárquicas (por ejemplo, arquitecturas de petición y retorno), la *complejidad estructural* de un módulo *i* se define de la forma siguiente:

$$S(i) = f^2_{\text{out}}(i)$$

donde $f_{\text{out}}(i)$ es el *fan-out*[6] del módulo *i*.

La *complejidad de datos* ofrece un indicio de la complejidad que hay en la interfaz interna para un módulo *i* y se define como

$$D(i) = \frac{v(i)}{f_{\text{out}}(i) + 1}$$

donde $v(i)$ es el número de variables de entrada y salida que pasan hacia y desde el módulo *i*.

Finalmente, la *complejidad del sistema* se define como la suma de las complejidades estructural y de datos y se especifica como

$$C(i) = S(i) + D(i)$$

Conforme aumenta el valor de cada una de estas complejidades, la complejidad arquitectónica global del sistema también aumenta. Esto conduce a una mayor probabilidad de que también aumenten el esfuerzo de integración y el de pruebas.

Fenton [Fen91] sugiere algunas métricas simples de morfología (es decir, de forma) que permiten la comparación de diferentes arquitecturas de programa usando un conjunto de dimensiones directas. Con respecto a la arquitectura de llamado y retorno de la figura 23.4, puede definirse la siguiente métrica:

Tamaño = $n + a$

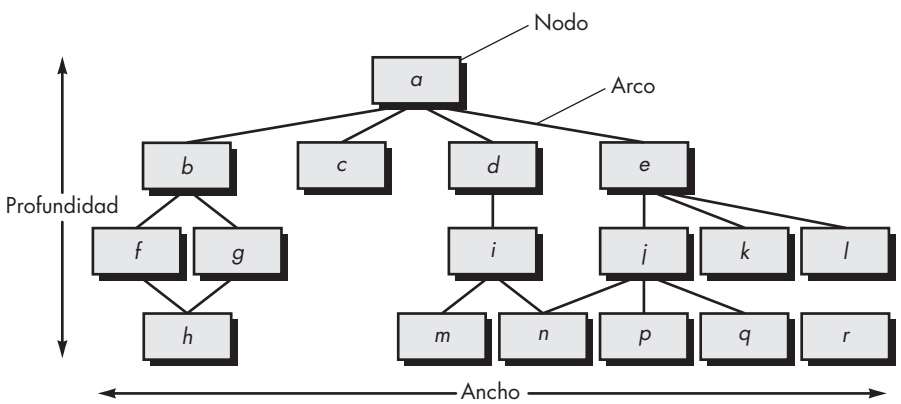
donde *n* es el número de nodos y *a* es el número de arcos. Para la arquitectura que se muestra en la figura 23.4,

Tamaño = 17 + 18 = 35
Profundidad = trayectoria más larga desde el nodo raíz (superior) hasta un nodo hoja. Para la arquitectura que se muestra en la figura 23.4, profundidad = 4.
Ancho = número máximo de nodos en cualquier nivel de la arquitectura. Para la arquitectura que se muestra en la figura 23.4, ancho = 6.

La razón arco a nodo, $r = a/n$, mide la densidad de conectividad de la arquitectura y puede proporcionar un indicio simple del acoplamiento de la arquitectura. Para la arquitectura que se muestra en la figura 23.4, $r = 18/17 = 1.06$.

La comandancia de los sistemas de la fuerza aérea estadounidense [USA87] desarrolló algunos indicadores de calidad del software que se basan en las características de diseño mensura-

FIGURA 23.4
Métricas de morfología



6 *Fan-out* (cargabilidad o abanico de salida) se define como el número de módulos inmediatamente subordinados al módulo *i*, es decir, el número de módulos que invoca directamente el módulo *i*.

bles de un programa de computadora. Al usar conceptos similares a los propuestos en IEEE Std. 982.1-1988 [IEE94], la fuerza aérea usa la información obtenida de los diseños de datos y arquitectónico para derivar un *índice de calidad de la estructura del diseño* (ICED) que varía de 0 a 1. Es necesario averiguar los siguientes valores para calcular el ICED [Cha89]:

S_1 = número total de módulos definidos en la arquitectura del programa
S_2 = número de módulos cuya función correcta depende de la fuente de entrada de datos o que produce los datos que se van a utilizar en alguna otra parte (en general, los módulos de control, entre otros, no se contarían como parte de S_2)
S_3 = número de módulos cuya función correcta depende del procesamiento previo
S_4 = número de ítems de base de datos (incluidos objetos de datos y todos los atributos que definen objetos)
S_5 = número total de ítems de base de datos únicos
S_6 = número de segmentos de base de datos (diferentes registros u objetos individuales)
S_7 = número de módulos con una sola entrada y salida (el procesamiento de excepción no se considera como una salida múltiple)

Cita:

"La medición puede verse como una desviación. Ésta es necesaria porque los humanos básicamente no son capaces de tomar decisiones claras y objetivas [sin apoyo cuantitativo]".

Horst Zuse

Una vez determinados los valores de S_1 a S_7 para un programa de cómputo, pueden calcularse los siguientes valores intermedios:

Estructura del programa: D_1, donde D_1 se define del modo siguiente: si el diseño arquitectónico se desarrolló usando un método distinto (por ejemplo, diseño orientado a flujo de datos o diseño orientado a objeto), entonces $D_1 = 1$, de otro modo $D_1 = 0$.

Independencia de módulo: $D_2 = 1 - \frac{S_2}{S_1}$

Módulos no dependientes del procesamiento previo: $D_3 = 1 - \frac{S_3}{S_1}$

Tamaño de base de datos: $D_4 = 1 - \frac{S_5}{S_4}$

Compartimentalización de base de datos: $D_5 = 1 - \frac{S_6}{S_4}$

Entrada/salida de módulo característico: $D_6 = 1 - \frac{S_7}{S_1}$

Con la determinación de estos valores intermedios, el ICED se calcula en la forma siguiente:

$$\text{ICED} = \Sigma\, w_i D_i$$

donde $i = 1$ a 6, w_i es el peso relativo de la importancia de cada uno de los valores intermedios y $\Sigma w_i = 1$ (si todos los D_i pesan igual, entonces $w_i = 0.167$).

El valor del ICED para diseños anteriores puede determinarse y compararse con un diseño que actualmente esté en desarrollo. Si el ICED es significativamente menor que el promedio, se indican más trabajo de diseño y revisión. De igual modo, si se hacen grandes cambios a un diseño existente puede calcularse el efecto de dichos cambios en el ICED.

23.3.2 Métricas para diseño orientado a objetos

Hay mucho de subjetivo en el diseño orientado a objetos: un diseñador experimentado "sabe" cómo caracterizar un sistema OO de modo que implemente de manera efectiva los requerimientos del cliente. Pero, conforme un modelo de diseño OO crece en tamaño y complejidad, una visión más objetiva de las características del diseño puede beneficiar tanto al diseñador experimentado (quien adquiere comprensión adicional) como al novato (quien obtiene un indicio de la calidad que de otro modo no tendría disponible).

En un tratamiento detallado de las métricas de software para sistemas OO, Whitmire [Whi97] describe nueve características distintas y mensurables de un diseño OO:

¿Qué características pueden medirse cuando se valora un diseño OO?

Tamaño. El tamaño se define en función de cuatro visiones: población, volumen, longitud y funcionalidad. La *población* se mide al realizar un conteo estático de entidades OO, tales como clases u operaciones. Las medidas de *volumen* son idénticas a las medidas de población, pero se recolectan de manera dinámica: en un instante de tiempo determinado. La *longitud* es una medida de una cadena de elementos de diseño interconectados (por ejemplo, la profundidad de un árbol de herencia es una medida de longitud). Las métricas de *funcionalidad* proporcionan un indicio indirecto del valor entregado al cliente por una aplicación OO.

Cita:

"Muchas de las decisiones en las cuales tuve que confiar en el folclore y el mito, ahora se pueden resolver haciendo uso de datos cuantitativos."

Scott Whitmire

Complejidad. Como el tamaño, existen muchas visiones diferentes de la complejidad del software [Zus97]. Whitmire ve la complejidad en términos de características estructurales al examinar cómo se relacionan mutuamente las clases de un diseño OO.

Acoplamiento. Las conexiones físicas entre elementos del diseño OO (por ejemplo, el número de colaboraciones entre clases o el de mensajes que pasan entre los objetos) representan el acoplamiento dentro de un sistema OO.

Suficiencia. Whitmire define *suficiencia* como "el grado en el que una abstracción posee las características requeridas de él o en el que un componente de diseño posee características en su abstracción, desde el punto de vista de la aplicación actual". Dicho de otra forma, se pregunta: "¿Qué propiedades debe poseer esta abstracción (clase) para serme útil?" [Whi97]. En esencia, un componente de diseño (por ejemplo, una clase) es *suficiente* si refleja por completo todas las propiedades del objeto de dominio de aplicación que se modela, es decir, si la abstracción (clase) posee sus características requeridas.

Completitud. La única diferencia entre completitud y suficiencia es "el conjunto de características contra las cuales se compara la abstracción o el componente de diseño" [Whi97]. La suficiencia compara la abstracción desde el punto de vista de la aplicación actual. La *completitud* considera múltiples puntos de vista, y plantea la pregunta: "¿qué propiedades se requieren para representar por completo al objeto de dominio problema?". Puesto que el criterio para completitud considera diferentes puntos de vista, tiene una implicación indirecta en el grado en el que puede reutilizarse la abstracción o el componente de diseño.

Cohesión. Como su contraparte en software convencional, un componente OO debe diseñarse de manera que tenga todas las operaciones funcionando en conjunto para lograr un solo propósito bien definido. La cohesividad de una clase se determina al examinar el grado en el que "el conjunto de propiedades que posee es parte del problema o dominio de diseño" [Whi97].

Primitivismo. Una característica que es similar a la simplicidad, el primitivismo (aplicado tanto a operaciones como a clases), es el grado en el que una operación es atómica, es decir, la operación no puede construirse a partir de una secuencia de otras operaciones contenidas dentro de una clase. Una clase que muestra un alto grado de primitivismo encapsula sólo operaciones primitivas.

Similitud. El grado en el que dos o más clases son similares en términos de su estructura, función, comportamiento o propósito se indica mediante esta medida.

Volatilidad. Como se menciona muchas veces en este libro, los cambios en el diseño pueden ocurrir cuando se modifican los requerimientos o cuando ocurren modificaciones en otras partes de una aplicación, lo que da como resultado la adaptación obligatoria del componente de diseño en cuestión. La volatilidad de un componente de diseño OO mide la probabilidad de que ocurrirá un cambio.

En realidad, las métricas de producto para sistemas OO pueden aplicarse no sólo al modelo de diseño, sino también al de requerimientos. En las siguientes secciones se estudian las métricas

que proporcionan un indicio de la calidad en el nivel de clase OO y en el de operación. Además, también se exploran las métricas aplicables a la administración del proyecto y a las pruebas.

23.3.3 Métricas orientadas a clase: la suite de métricas CK

La clase es la unidad fundamental de un sistema OO. Por tanto, las medidas y métricas para una clase individual, la jerarquía de clase y las colaboraciones de clase serán invaluables cuando se requiera valorar la calidad del diseño OO. Una clase encapsula datos y la función que los manipula. Con frecuencia es el "padre" de las subclases (en ocasiones llamadas hijos) que heredan sus atributos y operaciones. Usualmente colabora con otras clases. Cada una de estas características puede usarse como la base para la medición.[7]

Chidamber y Kemerer propusieron uno de los conjuntos de métricas de software OO de mayor referencia [Chi94]. En ocasiones llamada *suite de métricas CK*, los autores proponen seis métricas de diseño basadas en clase para sistemas OO.[8]

Métodos ponderados por clase (MPC). Suponga que n métodos de complejidad c_1, c_2, ..., c_n se definen para una clase **C**. La métrica de complejidad específica que se elige (por ejemplo, complejidad ciclomática) debe normalizarse de modo que la complejidad nominal para un método tome un valor de 1.0.

$$\text{MPC} = \Sigma c_i$$

para $i = 1$ hasta n. El número de métodos y su complejidad son indicadores razonables de la cantidad de esfuerzo requerido para implementar y probar una clase. Además, mientras más grande sea el número de métodos, más complejo será el árbol de herencia (todas las subclases heredan los métodos de sus padres). Finalmente, conforme el número de métodos crece para una clase determinada, es probable que se vuelva cada vez más específica su aplicación y, por tanto, limite la potencial reutilización. Por todas estas razones, MPC debe mantenerse tan bajo como sea razonable.

Aunque parecería relativamente directo desarrollar un conteo para el número de métodos en una clase, el problema en realidad es más complejo de lo que parece. Debe desarrollarse un enfoque de conteo consistente [Chu95].

Profundidad del árbol de herencia (PAH). Esta métrica es "la máxima longitud desde el nodo hasta la raíz del árbol" [Chi94]. En la figura 23.5, el valor de la PAH para la jerarquía de clase que se muestra es 4. Conforme crece la PAH, es probable que las clases de nivel inferior hereden muchos métodos. Esto conduce a potenciales dificultades cuando se intenta predecir el comportamiento de una clase. Una jerarquía de clase profunda (PAH grande) también conduce a mayor complejidad de diseño. En el lado positivo, grandes valores de PAH implican que muchos métodos pueden reutilizarse.

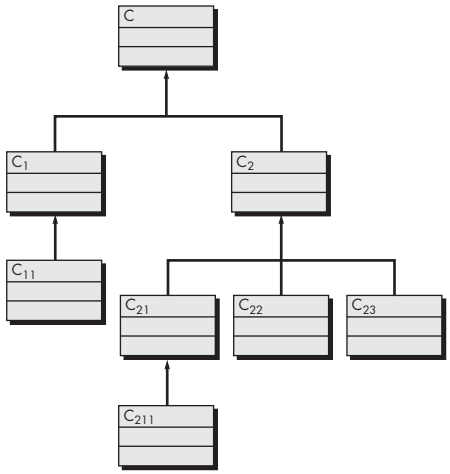
Número de hijos (NDH). Las subclases que son inmediatamente subordinadas a una clase en la jerarquía de clase se denominan hijos. En la figura 23.5, la clase $\mathbf{C_2}$ tiene tres hijos: subclases $\mathbf{C_{21}}$, $\mathbf{C_{22}}$ y $\mathbf{C_{23}}$. Conforme crece el número de hijos, el reuso aumenta, pero también, como el NDH aumenta, la abstracción representada por la clase padre puede diluirse si algunos de los hijos no son miembros adecuados de la clase padre. Conforme el NDH aumenta, la cantidad de pruebas (requeridas para ejercitar cada hijo en su contexto operativo) también aumentará.

---

7 Debe observarse que, en la literatura técnica, actualmente está en debate la validez de algunas de las métricas estudiadas en este capítulo. Quienes defienden la teoría de mediciones demandan un grado de formalismo que algunas métricas OO no proporcionan. Sin embargo, es razonable afirmar que las métricas mencionadas ofrecen comprensión útil para el ingeniero de software.

8 Chidamber, Darcy y Kemerer usan el término *métodos* en lugar de *operaciones*. El uso de este término se refleja en esta sección.

FIGURA 23.5
Una jerarquía de clase





Los conceptos de acoplamiento y cohesión se aplican tanto a software convencional como a OO. Mantenga el acoplamiento de clase bajo y la cohesión de clase y operación alta.

Acoplamiento entre clases de objetos (ACO). El modelo CRC (capítulo 6) puede usarse para determinar el valor para el ACO. En esencia, ACO es el número de colaboraciones citadas para una clase en su tarjeta índice CRC.[9] Conforme el ACO aumenta, es probable que la reusabilidad de una clase disminuya. Valores altos de ACO también complican las modificaciones y las pruebas que sobrevienen cuando se realizan modificaciones. En general, los valores de ACO para cada clase deben mantenerse tan bajos como sea razonable. Esto es consistente con el lineamiento general para reducir el acoplamiento en el software convencional.

Respuesta para una clase (RPC). Respuesta para una clase es "un conjunto de métodos que potencialmente pueden ejecutarse en respuesta a un mensaje recibido por un objeto de dicha clase" [Chi94]. RPC es el número de métodos en el conjunto respuesta. Conforme aumenta la RPC, también lo hace el esfuerzo requerido para probar, pues la secuencia de pruebas (capítulo 19) crece. Igualmente, se sigue que, conforme la RPC aumenta, la complejidad de diseño global de la clase aumenta.

Falta de cohesión en métodos (FCOM). Cada método dentro de una clase **C** accede a uno o más atributos (también llamados variables de instancia). FCOM es el número de métodos que acceden a uno o más de los mismos atributos.[10] Si ningún método accede a los mismos atributos, entonces la FCOM = 0. Para ilustrar el caso donde la FCOM ≠ 0, considere una clase con seis métodos. Cuatro de ellos tienen uno o más atributos en común (es decir, acceden a atributos comunes). Por tanto, la FCOM = 4. Si la FCOM es alta, los métodos pueden acoplarse unos con otros mediante atributos. Esto aumenta la complejidad del diseño de clase. Aunque hay casos en los que un valor alto de la FCOM es justificable, es deseable mantener alta la cohesión, es decir, mantener baja la FCOM.[11]

9 Si las tarjetas índice CRC se desarrollan manualmente, completitud y consistencia deben valorarse antes de que el ACO pueda determinarse de manera confiable.

10 La definición formal es un poco más compleja. Vea [Chi94] para detalles.

11 La métrica FCOM proporciona útil comprensión en algunas situaciones, pero puede confundir en otras. Por ejemplo, mantener en acoplamiento encapsulado dentro de una clase aumenta la cohesión del sistema como un todo. Por tanto, en al menos un sentido importante, la FCOM más alta en realidad sugiere que una clase puede tener mayor cohesión, no menor.

CasaSegura



Aplicación de métricas CK

La escena: Cubículo de Vinod.

Participantes: Vinod, Jamie, Shakira y Ed, miembros del equipo de ingeniería del software *CasaSegura*, quienes continúan trabajando en el diseño en el nivel de componentes y en el diseño de casos de prueba.

La conversación:

Vinod: ¿Alguno de ustedes tuvo oportunidad de leer la descripción de la suite de métricas CK que envié el miércoles e hizo las mediciones?

Shakira: No fue muy complicado. Regresé a mi clase UML y a diagramas de secuencia, como sugeriste, y obtuve conteos burdos para PAH, RPC y FCOM. No pude encontrar el modelo CRC, así que no conté ACO.

Jamie (sonríe): No pudiste encontrar el modelo CRC porque yo lo tengo.

Shakira: Por eso adoro a este equipo: comunicación soberbia.

Vinod: Yo hice mis conteos... ¿ustedes desarrollaron números para las métricas CK?

[Jamie y Ed asienten.]

Jamie: Dado que yo tenía las tarjetas CRC, eché un vistazo al ACO y parecía bastante uniforme a través de la mayoría de las clases. Hubo una excepción, la cual anoté.

Ed: Hay algunas clases donde la RPC es muy alta, en comparación con los promedios... tal vez debamos echar un vistazo para simplificarlos.

Jamie: Quizá sí, quizá no. Todavía estoy preocupado por el tiempo, y no quiero componer cosas que en realidad no están rotas.

Vinod: Estoy de acuerdo. Tal vez debas buscar clases que tengan malos números en al menos dos o más de las métricas CK. Cuestión de dos strikes y estás modificado.

Shakira [observa la lista de clases de Ed con alta RPC]: Mira, ve estas clases, tienen alta FCOM así como alta RPC. ¿Dos strikes?

Vinod: Sí, eso creo... será difícil implementar debido a la complejidad y dificultad para probar por la misma razón. Probablemente valdría la pena diseñar dos clases separadas para lograr el mismo comportamiento.

Jamie: ¿Crees que modificarlo nos ahorrará tiempo?

Vinod: A largo plazo, sí.

23.3.4 Métricas orientadas a clase: La suite de métricas MOOD

Cita:

"Analizar el software OO para evaluar su calidad se ha vuelto cada vez más importante conforme el paradigma [OO] continúa aumentando en popularidad."

Rachel Harrison *et al.*

Harrison, Counsell y Nithi [Har98b] proponen un conjunto de métricas para diseño orientado a objeto que proporciona indicadores cuantitativos para características de diseño OO. A continuación se presenta un muestreo de métricas MOOD.

Factor de herencia de método (FHM). El grado en el que la arquitectura de clase de un sistema OO utiliza la herencia tanto para métodos (operaciones) como para atributos se define como

$$\text{FHM} = \frac{\Sigma M_i\,(C_i)}{\Sigma M_a\,(C_i)}$$

donde la suma ocurre sobre $i = 1$ hasta TC. TC se define como el número total de clases en la arquitectura, C_i es una clase dentro de la arquitectura y

$$M_a(C_i) = M_d(C_i) + M_i(C_i)$$

donde

$M_a(C_i)$ = número de métodos que pueden invocarse en asociación con C_i
$M_d(C_i)$ = número de métodos declarados en la clase C_i
$M_i(C_i)$ = número de métodos heredados (y no invalidados) en C_i

El valor de FHM [el factor de herencia de atributo (FHA) se define de forma análoga] ofrece un indicio del impacto de la herencia sobre el software OO.

Factor de acoplamiento (FA). Anteriormente, en este capítulo, se dijo que el acoplamiento es un indicio de las conexiones entre elementos del diseño OO. La suite de métricas MOOD define el acoplamiento en la forma siguiente:

$$\text{FA} = \Sigma_i \Sigma_j\, is_client\, \frac{(C_i, C_j)}{T_c^2 - T_c}$$

donde las sumas ocurren sobre $i = 1$ hasta T_c y $j = 1$ hasta T_c. La función

is_client (es cliente) = 1, si y sólo si existe una relación entre la clase cliente C_c y la clase servidor C_s, y $C_c \neq C_s$
= 0, de otro modo

Aunque muchos factores afectan la complejidad, comprensibilidad y mantenimiento del software, es razonable concluir que, conforme el valor de FA aumenta, la complejidad del software OO también aumentará, y como resultado pueden sufrir la comprensibilidad, el mantenimiento y el potencial de reuso.

Harrison *et al.* [Har98b] presentan un análisis detallado de FHM y FA junto con otras métricas, y examinan su validez para usarlas en la valoración de la calidad del diseño.

23.3.5 Métricas OO propuestas por Lorenz y Kidd

En su libro acerca de métricas OO, Lorenz y Kidd [Lor94] dividen las métricas basadas en clase en cuatro amplias categorías; cada una tiene una relación en el diseño en el nivel de componentes: tamaño, herencia, internos y externos. Las métricas orientadas a tamaño para una clase de diseño OO se enfocan en conteos de atributos y operaciones para una clase individual y en valores promedio para el sistema OO como un todo. Las métricas basadas en herencia se enfocan en la forma en la que las operaciones se reutilizan a lo largo de la jerarquía de clases. Las métricas para interiores de clase se fijan en la cohesión (sección 23.3.3) y en los conflictos orientados a código. Y las métricas externas examinan el acoplamiento y el reuso. Un ejemplo de las métricas propuestas por Lorenz y Kidd es:

Tamaño de clase (TDC). El tamaño global de una clase puede determinarse usando las siguientes medidas:

- El número total de operaciones (tanto heredadas como operaciones de instancia privada) que se encapsulan dentro de la clase.
- El número de atributos (tanto heredados como de instancia privada) que encapsula la clase.



Durante la revisión del modelo de análisis, las tarjetas índice CRC proporcionarán un indicio razonable de los valores esperados para TDC. Si encuentra una clase con un gran número de responsabilidades, considere dividirla.

La métrica MPC propuesta por Chidamber y Kemerer (sección 23.3.3) también es una medida ponderada del tamaño de clase. Como se indicó anteriormente, grandes valores para TDC indican que una clase puede tener demasiada responsabilidad. Esto reducirá la reutilización de la clase y complicará la implementación y las pruebas. En general, las operaciones y atributos heredados o públicos deben ponderarse más para determinar el tamaño de clase [Lor94]. Las operaciones y atributos privados permiten la especialización y están más localizadas en el diseño. También pueden calcularse promedios para el número de atributos y operaciones de clase. Mientras más bajos sean los valores promedio para el tamaño, hay más probabilidad de que las clases dentro del sistema puedan reutilizarse ampliamente.

23.3.6 Métricas de diseño en el nivel de componente

Las métricas de diseño en el nivel de componente para componentes de software convencional se enfocan en las características internas de un componente de software e incluyen medidas de cohesión de módulo, acoplamiento y complejidad. Dichas medidas pueden ayudarlo a juzgar la calidad de un diseño en el nivel de componente.

Las métricas de diseño en el nivel de componente pueden aplicarse una vez desarrollado el diseño procedural y son "cajas de cristal" en tanto requieren conocimiento del funcionamiento interior del módulo en el que se está trabajando. Alternativamente, pueden demorarse hasta que el código fuente esté disponible.

PUNTO CLAVE
Es posible calcular medidas de la independencia funcional (acoplamiento y cohesión) de un componente y usarlas para valorar la calidad de un diseño.

Métricas de cohesión. Bieman y Ott [Bie94] definen una colección de métricas que proporcionan un indicio de la cohesión (capítulo 8) de un módulo. Las métricas se definen mediante cinco conceptos y medidas:

Rebanada de datos (*data slice*). Dicho de manera simple, una rebanada de datos es una marcha hacia atrás a través de un módulo que busca valores de datos que afecten la ubicación del módulo donde comenzó la marcha. Debe observarse que es posible definir tanto las rebanadas de programa (que se enfocan en enunciados y condiciones) como las rebanadas de datos.

Símbolos de datos (*data tokens*). Las variables definidas por un módulo pueden definirse como *tokens* de datos para el módulo.

Símbolos pegamento (*glue tokens*). Este conjunto de *tokens* de datos se encuentra en una o más rebanadas de datos.

Símbolos superpegamento (*superglue tokens*). Estos *tokens* de datos son comunes a cada rebanada de datos en un módulo.

Pegajosidad (*stickiness*). La pegajosidad relativa de un *token* pegamento es directamente proporcional al número de rebanadas de datos que enlaza.

Bieman y Ott desarrollan métricas para *cohesión funcional fuerte* (CFF), *cohesión funcional débil* (CFD) y *adhesividad* (el grado relativo en el que los *tokens* pegamento enlazan rebanadas de datos). Un análisis detallado de las métricas de Bieman y Ott, se deja a los autores [Bie94].

Métricas de acoplamiento. El acoplamiento de módulo proporciona un indicio de cuán "conectado" está un módulo con otros módulos, con datos globales y con el entorno exterior. En el capítulo 9 se estudió el acoplamiento en términos cualitativos.

Dhama [Dha95] propuso una métrica para acoplamiento de módulo que abarca acoplamiento de datos y flujo de control, acoplamiento global y acoplamiento ambiental. Las medidas requeridas para calcular acoplamiento de módulo se definen en función de cada uno de los tres tipos de acoplamiento anotados anteriormente.
Para acoplamiento de datos y flujo de control,

d_i = número de parámetros de datos de entrada
c_i = número de parámetros de control de entrada
d_o = número de parámetros de datos de salida
c_o = número de parámetros de control de salida

Para acoplamiento global,

g_d = número de variables globales usadas como datos
g_c = número de variables globales usadas como control

Para acoplamiento de entorno,

w = número de módulos llamados (*fan-out*)
r = número de módulos que llaman al módulo bajo consideración (*fan-in*)

Al usar estas medidas, un indicador de acoplamiento de módulo m_c se define de la forma siguiente:

$$m_c = \frac{k}{M}$$

donde k es una constante de proporcionalidad y

$$M = d_i + (a \times c_i) + d_o + (b \times c_o) + g_d + (c \times g_c) + w + r$$

Los valores para k, a, b y c deben derivarse de manera empírica.

Conforme el valor de m_c crece, el acoplamiento de módulo global disminuye. Con la finalidad de que la métrica de acoplamiento se mueva hacia arriba conforme el grado de acoplamiento aumenta, una métrica de acoplamiento revisada puede definirse como

$$C = 1 - m_c$$

donde el grado de acoplamiento aumenta conforme el valor de M aumenta.

Métricas de complejidad. Para determinar la complejidad del flujo de control de programa, pueden calcularse varias métricas de software. Muchas de éstas se basan en el gráfico de flujo. Un gráfico (capítulo 18) es una representación compuesta de nodos y ligas (también llamadas aristas). Cuando las ligas (aristas) se dirigen, el gráfico de flujo es un gráfico dirigido.

McCabe y Watson [McC94] identifican algunos usos importantes para las métricas de complejidad:

> Las métricas de complejidad pueden usarse para predecir información crucial acerca de la confiabilidad y el mantenimiento de los sistemas de software a partir de análisis automáticos de código fuente [o información de diseño procedimental]. Las métricas de complejidad también proporcionan retroalimentación durante el proyecto de software para ayudar a controlar la [actividad de diseño]. Durante las pruebas y el mantenimiento, proporcionan información detallada acerca de los módulos de software para ayudar a destacar áreas de potencial inestabilidad.

La métrica de complejidad para software de computadora más ampliamente usada es la complejidad ciclomática, originalmente desarrollada por Thomas McCabe [McC76] y que se estudió en detalle en el capítulo 18.

PUNTO CLAVE
La complejidad ciclomática es sólo una entre un gran número de métricas de complejidad.

Zuse ([Zus90], [Zus97]) presenta una discusión enciclopédica de no menos de 18 diferentes categorías de métricas de complejidad de software. El autor expone las definiciones básicas para las métricas en cada categoría (por ejemplo, existen algunas variaciones en la métrica de complejidad ciclomática) y luego analiza y critica cada una. El trabajo de Zuse es el más exhaustivo publicado a la fecha.

23.3.7 Métricas orientadas a operación

Puesto que la clase es la unidad dominante en los sistemas OO, se han propuesto menos métricas para operaciones que residen dentro de una clase. Churcher y Shepperd [Chu95] analizan esto cuando afirman: "Los resultados de estudios recientes indican que los métodos tienden a ser pequeños, tanto en términos de número de enunciados como en complejidad lógica [Wil93], lo que sugiere que la estructura de conectividad de un sistema puede ser más importante que el contenido de los módulos individuales." Sin embargo, puede obtenerse algo de comprensión al examinar las características promedio para los métodos (operaciones). Tres métricas simples, propuestas por Lorenz y Kidd [Lor94], son apropiadas:

Tamaño promedio de operación (TO_{prom}). El tamaño puede determinarse al contar el número de líneas de código o el de mensajes enviados por la operación. Conforme aumenta el número de mensajes enviados por una sola operación, es probable que las responsabilidades no se hayan asignado bien dentro de una clase.

Complejidad de la operación (CO). La complejidad de una operación puede calcularse usando cualquiera de las métricas de complejidad propuestas para software convencional [Zus90]. Puesto que las operaciones deben limitarse a una responsabilidad específica, el diseñador debe luchar por mantener la CO tan baja como sea posible.

Número promedio de parámetros por operación (NP_{prom}). Mientras más grande sea el número de parámetros de operación, más compleja es la colaboración entre objetos. En general, el NP_{prom} debe mantenerse tan bajo como sea posible.

23.3.8 Métricas de diseño de interfaz de usuario

Cita:

"Es posible aprender al menos un principio del diseño de interfaz de usuario al cargar una lavadora de platos. Si apila muchos en ella, nada quedará muy limpio."

Autor desconocido

Aunque hay considerable literatura acerca del diseño de interfaces hombre/computadora (capítulo 11), se ha publicado relativamente poca información acerca de las métricas que proporcionarían comprensión de la calidad y de la usabilidad de la interfaz.

Sears [Sea93] sugiere que la *corrección de la plantilla* (CP) es una métrica de diseño valioso para las interfaces hombre/computadora. Una GUI típica usa entidades de plantilla (íconos gráficos, texto, menús, ventanas y similares) para auxiliar al usuario a completar tareas. Para lograr una tarea dada usando una GUI, el usuario debe moverse de una entidad de plantilla a la siguiente. La posición absoluta y relativa de cada entidad de plantilla, la frecuencia con la que se usa y el "costo" de la transición desde una entidad de plantilla a la siguiente contribuirán a la corrección de la interfaz.

Un estudio de métricas de página web [Ivo01] indica que las características simples de los elementos de la plantilla también pueden tener un impacto significativo sobre la calidad percibida del diseño GUI. El número de palabras, vínculos, gráficos, colores y fuentes (entre otras características) contenidas en una página web afectan la complejidad percibida y la calidad de dicha página.

Es importante observar que la selección de un diseño GUI puede guiarse con métricas como CP, pero el árbitro final debe ser la entrada del usuario con base en los prototipos GUI. Nielsen y Levy [Nie94] reportan que "uno tiene una oportunidad razonablemente grande de triunfar si se elige entre [diseños de] interfaz basadas exclusivamente en opiniones de los usuarios. El rendimiento de tarea promedio de los usuarios y su satisfacción subjetiva con una GUI están enormemente correlacionados".

23.4 MÉTRICAS DE DISEÑO PARA *WEBAPPS*

Un útil conjunto de medidas y métricas para *webapps* proporciona respuestas cuantitativas a las siguientes preguntas:

- ¿La interfaz de usuario promueve usabilidad?
- ¿La estética de la *webapp* es apropiada para el dominio de aplicación y agrada al usuario?
- ¿El contenido se diseñó de tal forma que imparte más información con menos esfuerzo?
- ¿La navegación es eficiente y directa?
- ¿La arquitectura de la *webapp* se diseñó para alojar las metas y objetivos especiales de los usuarios de la *webapp*, la estructura de contenido y funcionalidad, y el flujo de navegación requerido para usar el sistema de manera efectiva?
- ¿Los componentes se diseñaron de manera que se reduce la complejidad procedimental y se mejora la exactitud, la confiabilidad y el desempeño?

CONSEJO

Muchas de estas métricas son aplicables a todas las interfaces de usuario y deben considerarse en conjunto con las que se presentaron en la sección 23.3.8.

En la actualidad, cada una de estas preguntas puede abordarse sólo de manera cualitativa, porque todavía no existe una suite validada de métricas que proporcionen respuestas cuantitativas.

En los siguientes párrafos se presenta una muestra representativa de métricas de diseño *webapp* que se han propuesto en la literatura. Es importante observar que muchas de ellas todavía no se validan, por lo que deben usarse juiciosamente.

Métricas de interfaz. Para *webapps* pueden considerarse las siguientes medidas:

Métrica sugerida	*Descripción*
Corrección de plantilla	Ver sección 23.3.8
Complejidad de plantilla	Número de regiones[12] distintas definidas por una interfaz
Complejidad de región de plantilla	Número promedio de distintos vínculos por región
Complejidad de reconocimiento	Número promedio de distintos ítems que el usuario debe buscar antes de realizar una navegación o decidir la entrada de datos
Tiempo de reconocimiento	Tiempo promedio (en segundos) que tarda un usuario en seleccionar la acción adecuada para una tarea determinada
Esfuerzo de escritura	Número promedio de golpes de tecla requeridos para una función específica
Esfuerzo de toma de ratón	Número promedio de tomas de ratón por función
Complejidad de selección	Número promedio de vínculos que pueden seleccionarse por página
Tiempo de adquisición de contenido	Número promedio de palabras de texto por página web
Carga de memoria	Número promedio de distintos ítems de datos que el usuario debe recordar para lograr un objetivo específico

Métricas estéticas (diseño gráfico). Por su naturaleza, el diseño estético se apoya en el juicio cualitativo y por lo general no es sensible a la medición ni a las métricas. Sin embargo, Ivory *et al.* [Ivo01] proponen un conjunto de medidas que pueden ser útiles para valorar el impacto del diseño estético:

Métrica sugerida	*Descripción*
Conteo de palabra	Número total de palabras que aparecen en una página
Porcentaje de texto de cuerpo	Porcentaje de palabras que son cuerpo frente a texto de despliegue (es decir, títulos)
% texto cuerpo enfatizado	Porción de texto de cuerpo que se enfatiza (por ejemplo, negrillas, mayúsculas)
Conteo de posicionamiento de texto	Cambios en posición de texto desde el alineado a la izquierda
Conteo de grupo de texto	Áreas de texto resaltadas con color, regiones con bordes, reglas o listas
Conteo de vínculos	Vínculos totales en una página
Tamaño de página	Bytes totales para la página, así como elementos, gráficos y hojas de estilo
Porcentaje gráfico	Porcentaje de bytes de página que son usados para gráficos
Conteo gráfico	Gráficos totales en una página (no incluye gráficos especificados en guiones, applets y objetos)
Conteo de color	Total de colores empleados
Conteo de fuente	Total de fuentes empleadas (es decir, tipo + tamaño + negrilla + itálica)

Métricas de contenido. Las métricas en esta categoría se enfocan en la complejidad del contenido y en los grupos de objetos de contenido que se organizan en páginas [Men01].

Métrica sugerida	*Descripción*
Espera de página	Tiempo promedio requerido para que una página se descargue a diferentes velocidades de conexión
Complejidad de página	Número promedio de tipos diferentes de medios usados en la página, no incluido el texto

12 Una región distinta es un área dentro de la plantilla de despliegue que logra cierto conjunto específico de funciones relacionadas (por ejemplo, una barra de menú, un despliegue gráfico estático, un área de contenido, un despliegue animado).

Complejidad gráfica	Número promedio de medios gráficos por página
Complejidad de audio	Número promedio de medios de audio por página
Complejidad de video	Número promedio de medios de video por página
Complejidad de animación	Número promedio de animaciones por página
Complejidad de imagen escaneada	Número promedio de imágenes escaneadas por página

Métricas de navegación. Las métricas en esta categoría abordan la complejidad del flujo de navegación [Men01]. En general, son útiles sólo para aplicaciones web estáticas, que no incluyen vínculos y páginas generados de manera dinámica.

Métrica sugerida	*Descripción*
Complejidad de vinculación de página	Número de vínculos por página
Conectividad	Número total de vínculos internos, no incluidos vínculos generados de manera dinámica
Densidad de conectividad	Conectividad dividida por conteo de página

Usar un subconjunto de las métricas sugeridas puede servir para derivar relaciones empíricas que permiten a un equipo de desarrollo *webapp* valorar la calidad técnica y predecir el esfuerzo con base en las estimaciones de complejidad estimadas. En esta área todavía queda mucho trabajo por hacer.

HERRAMIENTAS DE SOFTWARE



Métricas técnicas para webapps

Objetivo: Auxiliar a los ingenieros web a desarrollar métricas *webapp* significativas que proporcionen comprensión acerca de la calidad global de una aplicación.

Mecánica: La mecánica de las herramientas varía.

Herramientas representativas:[13]

Netmechanic Tools, desarrollada por Netmechanic (**www.netmechanic.com**), es una colección de herramientas que ayudan a mejorar el desempeño de sitios web y que se enfocan en conflictos específicos de implementación.

NIST Web Metrics Testbed, desarrollada por The National Institute of Standards and Technology (**zing.ncsl.nist.gov/WebTools/**), abarca la siguiente colección de útiles herramientas que están disponibles para descarga:

Web Static Analyzer Tool (WebSAT): comprueba el HTML de la página web contra lineamientos de usabilidad típicos.

Web Category Analysis Tool (WebCAT): permite que el ingeniero de usabilidad construya y realice un análisis de categoría web.

Web Variable Instrumenter Program (WebVIP): instrumenta un sitio web para capturar una bitácora de interacción con el usuario.

Framework for Logging Usability Data (FLUD): implementa un formateador de archivo y analizador gramatical para representación de las bitácoras de interacción del usuario.

VisVIP Tool: produce una visualización 3D de las rutas de navegación del usuario a través de un sitio web.

TreeDec: agrega auxiliares de navegación a las páginas de un sitio web.

23.5 MÉTRICAS PARA CÓDIGO FUENTE

La teoría de Halstead de la "ciencia del software" [Hal77] propuso las primeras "leyes" analíticas para el software de computadora.[14] Halstead asignó leyes cuantitativas al desarrollo de software

13 Las herramientas que se mencionan aquí no representan un respaldo, sino una muestra de las herramientas en esta categoría.

14 Debe observarse que las "leyes" de Halstead generaron gran controversia, y muchos creen que la teoría subyacente tiene fallas. Sin embargo, se ha realizado verificación experimental para lenguajes de programación seleccionados (por ejemplo, [Fel89]).

Cita:

"El cerebro humano sigue un conjunto de reglas más rígido [para desarrollo de algoritmos] del que se tiene conocimiento."

Maurice Halstead

de computadora, usando un conjunto de medidas primitivas que pueden derivarse después de generar el código o de que el diseño esté completo. Las medidas son:

n_1 = número de operadores distintos que aparecen en un programa
n_2 = número de operandos distintos que aparecen en un programa
N_1 = número total de ocurrencias de operador
N_2 = número total de ocurrencias de operando

Halstead usa estas medidas primitivas para desarrollar: expresiones para la longitud de programa global, volumen mínimo potencial para un algoritmo, volumen real (número de bits requeridos para especificar un programa), nivel del programa (una medida de complejidad del software), nivel del lenguaje (una constante para un lenguaje determinado) y otras características, como esfuerzo de desarrollo, tiempo de desarrollo e incluso el número proyectado de fallas en el software.

Halstead muestra que la longitud N puede estimarse

$$N = n_1 \log_2 n_1 + n_2 \log_2 n_2$$

y el volumen del programa puede definirse

$$V = N \log_2 (n_1 + n_2)$$

Debe observarse que V variará con el lenguaje de programación y representa el volumen de información (en bits) requerido para especificar un programa.

CONSEJO

Los operadores incluyen todo el flujo de constructos de control, condicionales y operaciones matemáticas. Los operandos abarcan todas las variables y constantes de programa.

Teóricamente, debe existir un volumen mínimo para un algoritmo particular. Halstead define una razón de volumen L como la razón del volumen de la forma más compacta de un programa al volumen del programa real. En realidad, L siempre debe ser menor que 1. En términos de medidas primitivas, la razón de volumen puede expresarse como

$$L = \frac{2}{n_1} \times \frac{n_2}{N_2}$$

El trabajo de Halstead es sensible a la verificación experimental y se ha llevado a cabo un gran trabajo para investigar la ciencia del software. Un análisis de este trabajo está más allá del ámbito de este libro. Para mayor información, vea [Zus90], [Fen91] y [Zus97].

23.6 MÉTRICAS PARA PRUEBAS

PUNTO CLAVE

Las métricas de prueba se ubican en dos amplias categorías: 1) métricas que intentan predecir el número probable de pruebas requeridas en varios niveles de prueba y 2) métricas que se enfocan en la cobertura de pruebas para un componente determinado.

Aunque se ha escrito mucho acerca de las métricas de software para pruebas (por ejemplo, [Het93]), la mayoría de las métricas proponen enfocarse en el proceso de las pruebas, no en las características técnicas de las pruebas en sí. En general, los examinadores deben apoyarse en las métricas de análisis, diseño y código para guiarlos en el diseño y la ejecución de los casos de prueba.

Las métricas del diseño arquitectónico proporcionan información acerca de la facilidad o dificultad asociada con las pruebas de integración (sección 23.3) y de la necesidad de software de pruebas especializado (por ejemplo, resguardos y controladores). La complejidad ciclomática (una métrica de diseño en el nivel de componente) yace en el centro de la prueba de ruta base, un método de diseño de casos de prueba que se presentó en el capítulo 18. Además, la complejidad ciclomática puede usarse para dirigirse a módulos como candidatos para prueba de unidad extensa. Los módulos con alta complejidad ciclomática tienen más probabilidad de ser proclives al error que los módulos donde su complejidad ciclomática es menor. Por esta razón, debe emplear esfuerzo por arriba del promedio para descubrir errores en tales módulos antes de que se integren en un sistema.

23.6.1 Métricas de Halstead aplicadas para probar

El esfuerzo de prueba puede estimarse usando métricas derivadas de las medidas de Halstead (sección 23.5). Al usar las definiciones para volumen de programa *V* y nivel de programa *PL*, el esfuerzo *e* de Halstead puede calcularse como

$$PL = \frac{1}{(n_1/2) \times (N_2/n_2)} \tag{23.2a}$$

$$e = \frac{V}{PL} \tag{23.2b}$$

El porcentaje de esfuerzo de prueba global que se va a asignar a un módulo *k* puede estimarse usando la siguiente relación:

$$\text{Porcentaje de esfuerzo de prueba } (k) = \frac{e(k)}{\Sigma e(i)} \tag{23.3}$$

donde $e(k)$ se calcula para el módulo *k* usando las ecuaciones (23.2), y la suma en el denominador de la ecuación (23.3) es la suma del esfuerzo de Halstead a través de todos los módulos del sistema.

23.6.2 Métricas para pruebas orientadas a objetos

Las métricas del diseño OO anotadas en la sección 23.3 proporcionan un indicio de la calidad del diseño. También ofrecen un indicio general de la cantidad de esfuerzo de prueba requerido para ejercitar un sistema OO. Binder [Bin94b] sugiere un amplio arreglo de métricas de diseño que tienen influencia directa sobre la "comprobabilidad" de un sistema OO. Las métricas consideran aspectos de encapsulación y herencia.

Falta de cohesión en métodos (FCOM).[15] Mientras más alto sea el valor de la FCOM, más estados deben ponerse a prueba para garantizar que los métodos no generan efectos colaterales.



Las pruebas OO pueden ser bastante complejas. Las métricas pueden ayudarle a dirigir los recursos de prueba en hebras, escenarios y paquetes de clases que son "sospechosas" con base en las características medidas. Úselas.

Porcentaje público y protegido (PPP). Los atributos públicos se heredan de otras clases y, por tanto, son visibles para dichas clases. Los atributos protegidos son accesibles a los métodos en las subclases. Esta métrica indica el porcentaje de los atributos de clase que son públicos o protegidos. Valores altos de PPP aumentan la probabilidad de efectos colaterales entre las clases porque los atributos públicos y protegidos conducen a alto potencial para acoplamiento.[16] Las pruebas deben diseñarse para garantizar el descubrimiento de tales efectos colaterales.

Acceso público a miembros de datos (APD). Esta métrica indica el número de clases (o métodos) que pueden acceder a otros atributos de clase, una violación de la encapsulación. Valores altos de APD conducen al potencial de efectos colaterales entre clases. Las pruebas deben diseñarse para garantizar el descubrimiento de tales efectos colaterales.

Número de clases raíz (NCR). Esta métrica es un conteo de las distintas jerarquías de clase que se describen en el modelo de diseño. Deben desarrollarse las suites de prueba para cada clase raíz y la correspondiente jerarquía de clase. Conforme el NCR aumenta, también aumenta el esfuerzo de prueba.

***Fan-in* (FIN).** Cuando se usa en el contexto OO, el *fan-in* (abanico de entrada) en la jerarquía de herencia es un indicio de herencia múltiple. FIN > 1 indica que una clase hereda sus atributos y operaciones de más de una clase raíz. FIN > 1 debe evitarse cuando sea posible.

15 Vea la sección 23.3.3 para una descripción de FCOM.

16 Algunas personas promueven diseños sin que alguno de los atributos sea público o privado, es decir, PPP = 0. Esto implica que todos los atributos deben valorarse en otras clases mediante métodos.

Número de hijos (NDH) y profundidad del árbol de herencia (PAH).[17] Como se mencionó en el capítulo 19, los métodos de superclase tendrán que volverse a probar para cada subclase.

23.7 MÉTRICAS PARA MANTENIMIENTO

Todas las métricas de software presentadas en este capítulo pueden usarse para el desarrollo de nuevo software y para el mantenimiento del software existente. Sin embargo, se han propuesto métricas diseñadas explícitamente para actividades de mantenimiento.

IEEE Std. 982.1-1988 [IEE93] sugiere un *índice de madurez de software* (IMS) que proporcione un indicio de la estabilidad de un producto de software (con base en cambios que ocurran para cada liberación del producto). Para ello, se determina la siguiente información:

M_T = número de módulos en la liberación actual
F_c = número de módulos en la liberación actual que cambiaron
F_a = número de módulos en la liberación actual que se agregaron
F_d = número de módulos de la liberación anterior que se borraron en la liberación actual

El índice de madurez del software se calcula de la forma siguiente:

$$\text{IMS} = \frac{M_T - (F_a + F_c + F_d)}{M_T}$$

Conforme el IMS tiende a 1.0, el producto comienza a estabilizarse. El IMS también puede usarse como una métrica para planificar actividades de mantenimiento de software. El tiempo medio para producir una liberación de un producto de software puede correlacionarse con el IMS, y es posible desarrollar modelos empíricos para esfuerzo de mantenimiento.

HERRAMIENTAS DE SOFTWARE



Métricas de producto

Objetivo: Auxiliar a los ingenieros del software a desarrollar métricas significativas que valoren los productos operativos producidos durante el modelado de análisis y diseño, la generación de código fuente y las pruebas.

Mecánica: Las herramientas en esta categoría abarcan un amplio abanico de métricas y se implementan como una aplicación independiente o (más comúnmente) como funcionalidad que existe dentro de las herramientas para análisis y diseño, codificación o pruebas. En la mayoría de los casos, las herramientas de métricas analizan una representación del software (por ejemplo, un modelo UML o código fuente) y desarrollan como resultado una o más métricas.

Herramientas representativas:[18]

Krakatau Metrics, desarrollada por Power Software (**www.power-software.com/products**), calcula métricas de complejidad, Halstead y otras relacionadas para C/C++ y Java.

Metrics4C, desarrollada por +1 Software Engineering (**www.plus-one.com/Metrics4C_fact_sheet.html**), calcula una variedad de métricas arquitectónicas, de diseño y orientadas a código, así como métricas orientadas a proyecto.

Rational Rose, distribuida por IBM (**www.304.ibm.com/jct03001c/software/awdtools/developer/rose/**), es un amplio conjunto de herramientas para modelado UML que incorpora algunas características de análisis de métricas.

RSM, desarrollada por M-Squared Technologies (**msquaredtechnologies.com/m2rsm/index.html**), calcula una amplia variedad de métricas orientadas a código para C, C++ y Java.

Understand, desarrollada por Scientific Toolworks, Inc. (**www.scitools.com**), calcula métricas orientadas a código para varios lenguajes de programación.

17 Vea la sección 23.3.3 para una descripción del NDH y de la PAH.

18 Las herramientas que se mencionan aquí no representan un respaldo, sino una muestra de las herramientas en esta categoría. En la mayoría de los casos, los nombres de las herramientas son marcas registradas por sus respectivos desarrolladores.

23.8 Resumen

Las métricas de software proporcionan una forma cuantitativa para valorar la calidad de los atributos internos de producto y, por tanto, permiten valorar la calidad antes de construir el producto. Las métricas proporcionan la comprensión necesaria para crear modelos efectivos de requerimientos y diseño, código sólido y pruebas amplias.

Para ser útil en un contexto de mundo real, una métrica de software debe ser simple y calculable, convincente, congruente y objetiva. Debe ser independiente del lenguaje de programación y ofrecer retroalimentación efectiva.

Las métricas para el modelo de requerimientos se enfocan en los tres componentes del modelo: la función, los datos y el comportamiento. Las métricas para diseño consideran arquitectura, diseño en el nivel de componentes y conflictos en el diseño de interfaz. Las métricas de diseño arquitectónico consideran los aspectos estructurales del modelo de diseño. Las métricas de diseño en el nivel de componente proporcionan un indicio de la calidad del módulo al establecer medidas indirectas para cohesión, acoplamiento y complejidad. Las métricas de diseño de interfaz de usuario ofrecen un indicio de la facilidad con la que puede usarse una GUI. Las métricas *webapp* consideran aspectos de la interfaz de usuario, así como estética, contenido y navegación de la *webapp*.

Las métricas para los sistemas OO se enfocan en mediciones que pueden aplicarse a las características de clase y de diseño (localización, encapsulación, ocultamiento de información, herencia y técnicas de abstracción de objeto) que hacen única a la clase. La suite de métricas CK define seis métricas de software orientado a clase que se enfocan en la clase y en la jerarquía de clase. La suite de métricas también desarrolla métricas para valorar las colaboraciones entre clases y la cohesión en métodos que residen dentro de una clase. En un nivel orientado a clase, la suite de métricas CK puede aumentarse con las métricas propuestas por Lorenz y Kidd y con la suite de métricas MOOD.

Halstead proporciona un interesante conjunto de métricas en el nivel del código fuente. Al usar el número de operadores y operandos presentes en el código, la ciencia del software proporciona una variedad de métricas que pueden usarse para valorar la calidad del programa.

Pocas métricas de producto se han propuesto para uso directo en las pruebas de software y en el mantenimiento. Sin embargo, muchas otras métricas de producto pueden usarse para guiar el proceso de pruebas y como mecanismo para valorar la capacidad de mantenimiento de un programa de cómputo. Para valorar la comprobabilidad de un sistema OO, se ha propuesto una gran variedad de métricas OO.

Problemas y puntos por evaluar

23.1. La teoría de la medición es un tema avanzado que tiene un fuerte engranaje con las métricas de software. Con [Zus97], [Fen91], [Zus90] o fuentes en la web, escriba un breve ensayo que resalte las tesis principales de la teoría de la medición. Proyecto individual: desarrolle una presentación acerca del tema y preséntela en clase.

23.2. ¿Por qué no es posible desarrollar una sola métrica exhaustiva para la complejidad de programa o calidad de programa? Intente encontrar una medida o métrica de la vida diaria que viole los atributos de las métricas de software efectivos definidos en la sección 23.1.5.

23.3. Un sistema tiene 12 entradas externas, 24 salidas externas, presenta 30 diferentes consultas externas, gestiona 4 archivos lógicos internos y tiene interfaz con 6 diferentes sistemas legados (6 AIE). Todos estos datos son de complejidad promedio y el sistema global es relativamente simple. Calcule PF para el sistema.

23.4. El software para System X tiene 24 requerimientos funcionales individuales y 14 requerimientos no funcionales. ¿Cuál es la especificidad de los requerimientos y cuál es la completitud?

23.5. Un gran sistema de información tiene 1 140 módulos. Existen 96 módulos que realizan funciones de control y coordinación y 490 módulos cuya función depende del procesamiento previo. El sistema procesa aproximadamente 220 objetos de datos, cada uno de los cuales tiene un promedio de tres atributos. Existen 140 ítems de base de datos únicos y 90 diferentes segmentos de base de datos. Finalmente, 600 módulos tienen puntos de entrada y salida únicos. Calcule el ICED para este sistema.

23.6. Una clase **X** tiene 12 operaciones. La complejidad ciclomática se calcula para todas las operaciones en el sistema OO y el valor promedio de la complejidad de módulo es 4. Para la clase **X**, la complejidad para las operaciones de la 1 a la 12 es 5, 4, 3, 3, 6, 8, 2, 2, 5, 5, 4, 4, respectivamente. Calcule los métodos ponderados por clase.

23.7. Desarrolle una herramienta de software que calcule la complejidad ciclomática para un módulo de lenguaje de programación. Puede elegir el lenguaje.

23.8. Desarrolle una pequeña herramienta de software que realice análisis de Halstead sobre el código fuente del lenguaje de programación de su elección.

23.9. Un sistema legado tiene 940 módulos. La última liberación requirió el cambio de 90 de dichos módulos. Además, se agregaron 40 nuevos módulos y se removieron 12 módulos antiguos. Calcule el índice de madurez de software para el sistema.

Lecturas y fuentes de información adicionales

Existe un número sorprendentemente grande de libros que se dedican a las métricas de software, aunque la mayoría se enfocan en las métricas de proceso y proyecto, con la exclusión de métricas de producto. Lanza *et al.* (*Object-Oriented Metrics in Practice*, Springer, 2006) analizan métricas OO y su uso para valorar la calidad de un diseño. Genero (*Metrics for Software conceptual Models*, Imperial College Press, 2005) y Ejiogu (*Software Metrics*, BookSurge Publishing, 2005) presentan una amplia variedad de métricas técnicas para casos de uso, modelos UML y otras representaciones de modelado. Hutcheson (*Software Testing fundamentals: Methods and Metrics*, Wiley, 2003) presenta un conjunto de métricas para prueba. Kan (*Metrics and Models in Software Quality Engineering,* Addison-Wesley, 2a. ed., 2002), Fenton y Pfleeger (*Software Metrics: A Rigorous and Practical Approach,* Brooks-Cole Publishing, 1998), y Zuse [Zus97] escribieron tratamientos profundos de las métricas de producto.

Los libros de Card y Glass [Car90], Zuse [Zus90], Fenton [Fen91], Ejiogu [Eji91], Moeller y Paulish (*Software Metrics,* Chapman y Hall, 1993), y Hetzel [Het93] abordan métricas de producto con cierto detalle. Oman y Pfleeger (*Applying Software Metrics*, IEEE Computer Society Press, 1997) editaron una antología de importantes ensayos acerca de las métricas de software.

Ebert *et al.* (*Best Practices in Software Measurement*, Springer, 2004) consideran los métodos para establecer un programa de métricas y los principios subyacentes para la medición del software. Shepperd (*Foundations of Software Measurement*, Prentice-Hall, 1996) también aborda la teoría de medición con cierto detalle. La investigación actual se presenta en los *Proceedings of the Symposium on Software Metrics* (IEEE, publicación anual).

En [IEE93] se presenta un amplio resumen de decenas de métricas de software útiles. En general, un análisis de cada métrica se ha separado en las "primitivas" (medidas) esenciales requeridas para calcular la métrica y las relaciones apropiadas para efectuar el cálculo. Se proporciona análisis y muchas referencias en un apéndice.

Whitmire [Whi97] presenta un tratamiento amplio y matemáticamente sofisticado de las métricas OO. Lorenz y Kidd [Lor94] y Hendersen-Sellers (*Object-Oriented Metrics: Measures of Complexity*, Prentice-Hall, 1996) proporcionan tratamientos que se dedican a las métricas OO.

En internet, está disponible una gran variedad de fuentes de información acerca de las métricas del software. Una lista actualizada de referencias en la World Wide Web que son relevantes para la métrica del software puede encontrarse en el sitio del libro: **www.mhhe.com/engcs/compsci/pressman/professional/olc/ser.htm**